

AI 应用工程学习 APP | 56 讲内容开发素材

0. 总体判断

本 APP 的 56 个知识点不应该按“AI 名词百科”来组织，而应该按企业 AI 应用负责人的认知路径组织：

1. 先理解模型为什么能工作
2. 再理解模型为什么慢、贵、不稳定
3. 再理解模型如何平台化成 MaaS
4. 再理解 MaaS 如何支撑 Agent
5. 再理解 Agent 如何改造软件工程
6. 最后理解企业如何治理、评测、控成本和重塑组织

一句话定位：

这是一套面向企业 AI 应用负责人、AI 平台负责人和高级工程管理者们的“AI 应用工程教材”。

每一讲都必须帮助用户完成四个跃迁：

1. 从听过概念到能准确定义
2. 从知道术语到理解机制
3. 从理解机制到能诊断问题
4. 从诊断问题到能指导落地

1. 内容开发统一模板

每一讲统一按以下结构开发。

1.1 页面字段

```
{
  id: string;
  moduleId: string;
  order: number;
  title: string;
  difficulty: "basic" | "intermediate" | "advanced";
  estimatedMinutes: number;
  hasAnimation: boolean;
  tags: string[];

  oneSentence: string;
  whyItMatters: string;
  mentalModel: string;
  mechanism: string[];
  enterpriseCase: {
    title: string;
```

```
scenario: string;
problem: string;
analysis: string;
solution: string;
takeaway: string;
};
animationBrief?: {
  type: string;
  goal: string;
  steps: string[];
};
diagnosticQuestion: {
  scenario: string;
  question: string;
  options: string[];
  answer: string;
  explanation: string;
  troubleshootingPath: string[];
};
commonPitfalls: string[];
keyTakeaways: string[];
relatedConcepts: string[];
}
```

1.2 每一讲的标准章节

1. 一句话定义
2. 为什么重要
3. 心智模型
4. 机制拆解
5. 动画演示
6. 企业案例
7. 常见误区
8. 诊断题
9. 核心结论
10. 关联知识点

1.3 内容口径

不要写成科普文章。
不要写成论文综述。
不要写成提示词技巧合集。
不要写成产品营销文案。

应该写成：

面向企业 AI 工程负责人的工程教材。

每一讲都要回答：

1. 这个概念解决什么问题？
 2. 它在系统链路里处于哪一层？
 3. 出问题时会表现成什么现象？
 4. 应该看哪些指标？
 5. 应该怎么优化？
 6. 和 MaaS、Agent、RAG、软件工程闭环有什么关系？
-

2. 六大模块总览

模块 01：模型怎么工作，10 讲

目标：让用户理解大模型的基本机制和能力边界。

核心问题：

模型到底是怎么“看懂”和“生成”文本的？为什么它能推理？为什么它会幻觉？为什么上下文会影响结果？

模块 02：模型怎么跑得又快又稳，10 讲

目标：让用户理解推理系统、时延、吞吐、缓存、调度和算力瓶颈。

核心问题：

为什么同一个模型，有的系统快，有的系统慢？为什么高并发下会劣化？为什么 KV Cache、Batch、P-D 分离是关键？

模块 03：模型怎么变成企业平台，8 讲

目标：让用户理解 MaaS、模型网关、路由、缓存、成本和 SLA。

核心问题：

为什么企业不能只是接一个模型 API？为什么 MaaS 是成本、稳定性和治理控制面？

模块 04：模型怎么变成 Agent，12 讲

目标：让用户理解上下文工程、Agent Loop、工具调用、Skill 和多 Agent 协作。

核心问题：

为什么 Agent 不只是更长的 Prompt？为什么 Skill 会成为新的应用单元？为什么上下文工程是 Agent 成败关键？

模块 05: Agent 怎么改变软件工程，6 讲

目标：让用户理解 AI 原生软件工程的闭环模式。

核心问题：

需求、问题单、代码、测试、评审、运维，如何从人机交互变成人机协作？

模块 06: 企业怎么治理 AI，10 讲

目标：让用户理解评测、观测、安全、权限、成本、ROI 和组织阵型。

核心问题：

企业 AI 如何从“工具试用”走向“可治理、可复制、可考核的生产系统”？

3. 56 讲课程清单与内容素材

模块 01: 模型怎么工作

01. Token

难度：基础

建议时长：8 分钟

动画优先级：高

一句话定义：

Token 是模型处理文本的最小输入输出单位，不一定等于一个字或一个词。

这一讲要讲清楚：

1. 文本为什么要被切成 Token
2. Token 与字、词、字符的区别
3. 输入 Token 和输出 Token 如何影响成本
4. Token 为什么决定上下文长度
5. 企业应用中 Token 膨胀为什么会带来时延和成本问题

心智模型：

Token 是模型眼中的“文字积木”。用户看到自然语言，模型看到 Token 序列。

动画素材：

展示一句中文和一句英文如何被切成不同 Token；再展示 Token 进入模型，模型逐个生成输出 Token。

企业案例：

Agent 每轮都把完整历史、工具结果、大段文档塞入上下文，导致输入 Token 膨胀。用户感知是首字慢，平台侧表现为成本升高、TTFT 变长、缓存命中率下降。

诊断题方向：

一个业务 Agent 的输出变慢，但输出长度并没有增加，应该优先看输入 Token、上下文长度、工具结果注入量，而不是只看模型大小。

关联知识点：

上下文窗口、KV Cache、Token ROI、上下文压缩。

02. 词向量与语义空间

难度：基础

建议时长：8 分钟

动画优先级：中

一句话定义：

词向量是模型把 Token 映射到高维数字空间后的表示，使模型能够用数学方式处理语义关系。

这一讲要讲清楚：

1. 为什么模型不能直接处理文字
2. Token 如何变成向量
3. 语义相近为什么在向量空间里更接近
4. 向量表示如何支撑检索、分类和生成
5. 为什么向量相似不等于事实正确

心智模型：

词向量像“语义坐标”。每个 Token 都被放在一个高维地图上，距离近代表模型认为语义相近。

动画素材：

二维简化展示“猫、狗、汽车、服务器、模型”等词在空间中的相对位置。

企业案例：

RAG 中用户问“故障恢复”，检索系统可能召回“容灾切换”“服务自愈”“异常恢复”等语义接近内容，但不一定召回最新制度。

诊断题方向：

向量检索召回了语义相似但版本错误的文档，应该判断问题出在向量相似度、时效性过滤还是权限过滤。

关联知识点：

RAG、向量检索、混合检索、Answer Grounding。

03. Transformer

难度：基础

建议时长：10 分钟

动画优先级：中

一句话定义：

Transformer 是大语言模型的核心网络结构，依靠注意力机制在上下文中建立 Token 之间的关系。

这一讲要讲清楚：

1. Transformer 为什么适合处理序列
2. 它和传统循环网络的核心区别
3. 注意力、前馈网络、残差、归一化分别做什么
4. 多层堆叠为什么带来复杂表达能力
5. 企业应用中不需要推导公式，但要理解其性能代价

心智模型：

Transformer 像一个多层阅读器，每层都重新审视当前 Token 和上下文中其他 Token 的关系。

动画素材：

输入一句话，展示每层 Transformer 如何逐层加工 Token 表示。

企业案例：

同样上下文下，大模型能做复杂推理，小模型只能做表层匹配。原因不只是参数量，也包括模型结构、训练数据和推理能力差异。

诊断题方向：

一个小模型在知识问答上表现正常，但在多约束任务拆解上失败，应该判断是能力边界，而不是只调 Prompt。

关联知识点：

注意力机制、推理能力边界、模型路由、能力路由。

04. 注意力机制

难度：基础

建议时长：10 分钟

动画优先级：高

一句话定义：

注意力机制让模型在处理当前 Token 时，动态关注上下文中更相关的 Token。

这一讲要讲清楚：

1. 注意力不是人类意识，而是一种权重分配机制
2. 当前 Token 如何关注其他 Token
3. 为什么长上下文会增加计算和显存压力
4. 为什么上下文里的干扰信息会污染模型判断
5. 注意力机制如何解释“上下文工程”的重要性

心智模型：

注意力像阅读时的荧光笔。当前问题不同，被划重点的上下文也不同。

动画素材：

一句话中某个 Token 向其他 Token 连线，线条粗细表示注意力强弱。

企业案例：

用户问“这个接口为什么失败”，上下文里同时有旧日志、新日志、无关代码和历史讨论。模型可能关注错误内容，导致定位偏差。

诊断题方向：

模型回答引用了上下文中的旧结论，而不是最新日志，应排查上下文排序、优先级和污染问题。

关联知识点：

上下文污染、上下文优先级、RAG、Agent Trace。

05. 位置编码

难度：基础

建议时长：7 分钟

动画优先级：低

一句话定义：

位置编码让模型知道 Token 在序列中的相对或绝对位置，从而理解语序和上下文结构。

这一讲要讲清楚：

1. Transformer 本身不天然知道顺序
2. 为什么位置对语言理解很重要
3. 长上下文扩展为什么和位置机制有关
4. 位置混乱为什么会影响模型理解

5. 工程上如何通过结构化上下文降低位置负担

心智模型：

位置编码像给每个 Token 加上页码和坐标。

动画素材：

展示同样词语不同顺序导致含义变化，例如“模型 调用 工具”和“工具 调用 模型”。

企业案例：

Agent 上下文里需求、日志、代码、约束混在一起，模型很难判断优先级。结构化分段比简单堆文本更有效。

诊断题方向：

模型经常忽略前文约束，应检查上下文长度、约束位置、结构标签和系统提示词优先级。

关联知识点：

上下文窗口、系统提示词、上下文压缩。

06. 自回归生成

难度：基础

建议时长：8 分钟

动画优先级：高

一句话定义：

自回归生成是模型基于已有上下文一次预测一个下一个 Token 的生成方式。

这一讲要讲清楚：

1. 模型不是一次生成整篇答案
2. 每个输出 Token 都会成为下一步输入
3. 早期错误会影响后续生成
4. 为什么 Decode 阶段通常是逐 Token 慢速生成
5. 为什么流式输出能改善用户体验

心智模型：

模型像边写边想的作者，每写一个字，后面都基于已经写出的内容继续推。

动画素材：

展示模型从 Prompt 开始逐个生成 Token，每一步概率最高的候选被选中或采样。

企业案例：

Code Review Agent 一开始误判代码意图，后续建议都围绕错误假设展开。需要通过中间校验打断错误链路。

诊断题方向：

Agent 输出越写越偏，应判断是否需要分步校验、规划先行、草稿审查，而不是一次性长输出。

关联知识点：

采样策略、Agent Loop、工具调用、Human-in-the-loop。

07. 采样策略

难度：基础

建议时长：8 分钟

动画优先级：中

一句话定义：

采样策略决定模型在多个候选 Token 中如何选择下一个输出，从而影响确定性、创造性和稳定性。

这一讲要讲清楚：

1. Temperature 是什么
2. Top-p 和 Top-k 是什么
3. 低温更稳定，高温更多样
4. 不同任务应该使用不同采样策略
5. 企业场景通常优先稳定而非发散

心智模型：

采样像在候选答案池里选路。低温走最稳的路，高温允许探索更多岔路。

动画素材：

展示候选 Token 概率分布，调节 Temperature 后分布变平或变尖。

企业案例：

同一个 Issue 修复任务，多次运行结果差异很大。对于修复、评审、运维诊断类任务，应降低随机性并加强校验。

诊断题方向：

线上问答同一个问题多次答案差异过大，优先检查采样参数、系统提示词、上下文稳定性和模型版本。

关联知识点：

评测、回归集、质量门禁、模型版本治理。

08. 指令微调与偏好优化

难度：中级

建议时长：10 分钟

动画优先级：低

一句话定义：

指令微调和偏好优化让基础模型更会遵循人类任务指令，并输出更符合偏好的回答。

这一讲要讲清楚：

1. 预训练模型和指令模型的区别
2. 指令微调解决“听懂任务”的问题
3. 偏好优化解决“答案更符合人类偏好”的问题
4. 对齐不等于事实正确
5. 企业不能把模型礼貌等同于可靠

心智模型：

预训练让模型“会说话”，指令微调让模型“会听任务”，偏好优化让模型“更像人希望的回答方式”。

动画素材：

同一个问题经过基础模型、指令模型、偏好优化模型的输出差异对比。

企业案例：

模型回答很流畅、很礼貌，但引用了错误制度。说明回答风格好不代表答案可靠，仍需 Grounding 和引用验证。

诊断题方向：

用户反馈“答案看起来专业但事实错误”，应排查知识来源、检索、引用和评测，而不是只做提示词润色。

关联知识点：

幻觉、Answer Grounding、Eval、RAG。

09. 幻觉

难度：中级

建议时长：10 分钟

动画优先级：中

一句话定义：

幻觉是模型生成了看似合理但缺乏事实依据或与事实不一致的内容。

这一讲要讲清楚：

1. 幻觉不是系统故障，而是生成式模型的内在风险
2. 幻觉可能来自知识缺失、上下文不足、检索错误、任务歧义
3. 模型自信程度不等于事实可靠性
4. RAG、引用、工具校验可以降低幻觉
5. 企业场景必须把幻觉当成系统工程问题治理

心智模型：

模型像一个强语言补全器。它擅长补全合理文本，但不天然保证事实验证。

动画素材：

展示用户问题进入模型后，模型在“已有证据不足”情况下仍生成完整答案；再展示加入检索引用和工具校验后的变化。

企业案例：

运维 Agent 在没有实时监控数据时，编造了“可能是 CPU 高”的原因。正确做法是先调用监控工具，再基于证据诊断。

诊断题方向：

模型没有查到资料却给出确定答案，应检查是否要求引用、是否允许拒答、是否配置工具调用、是否有置信度策略。

关联知识点：

RAG、工具调用、Answer Grounding、AI 安全。

10. 推理能力边界

难度：中级

建议时长：12 分钟

动画优先级：低

一句话定义：

推理能力边界指模型在复杂规划、数学、代码、长链路任务和真实世界约束下的能力上限与失败模式。

这一讲要讲清楚：

1. 模型不是所有任务都能可靠完成
2. 单轮推理、多步推理、工具增强推理的区别
3. 小模型、大模型、专用模型的边界不同
4. 复杂任务需要拆解、校验和工具闭环
5. 企业应用要用系统设计弥补模型边界

心智模型：

模型能力像发动机功率。不是所有车都能拉重货，上坡还需要变速箱、刹车和导航。

动画素材：

展示一个复杂任务被拆成多个子任务：理解需求、查资料、写方案、执行、验证。

企业案例：

小模型可以回答 FAQ，但无法稳定完成跨系统问题单修复。应通过模型分级路由，让小模型处理简单任务，大模型处理复杂任务。

诊断题方向：

某任务在简单样例上成功，复杂样例上失败，应判断是模型能力、上下文不足、工具缺失还是流程设计问题。

关联知识点：

能力路由、多模型路由、Agent Loop、Eval。

模块 02：模型怎么跑得又快又稳

11. Prefill

难度：中级

建议时长：10 分钟

动画优先级：高

一句话定义：

Prefill 是模型处理输入上下文并生成首个输出前内部状态的阶段，通常决定首字时延的重要部分。

要讲清楚：

1. Prefill 处理完整输入上下文
2. 输入 Token 越多，Prefill 成本越高
3. Prefill 会生成 KV Cache
4. TTFT 与 Prefill 强相关
5. 高并发下 Prefill 会造成排队和资源竞争

动画素材：

展示长 Prompt 输入后，模型先“读全文”，写入 KV Cache，再生成第一个 Token。

企业案例：

RAG 问答把 20 篇文档全部塞入 Prompt，输出不长但 TTFT 很高。问题不在 Decode，而在 Prefill 输入过重。

诊断题方向：

首字慢但生成速度正常，优先排查输入 Token、Prefill 排队、上下文长度和缓存命中。

关联知识点：

TTFT、KV Cache、上下文压缩、RAG。

12. Decode

难度：中级

建议时长：10 分钟

动画优先级：高

一句话定义：

Decode 是模型逐个生成输出 Token 的阶段，通常决定长答案生成速度和吞吐。

要讲清楚：

1. Decode 是逐 Token 生成
2. Decode 阶段难以完全并行
3. 输出越长，Decode 时间越长
4. KV Cache 能减少重复计算
5. TPOT 是 Decode 体验的核心指标

动画素材：

展示模型每一步读取已有输出，生成下一个 Token，直到结束。

企业案例：

报告生成任务首字很快，但完整输出需要 2 分钟，说明瓶颈主要在 Decode 和输出长度控制。

诊断题方向：

TTFT 正常但长文生成慢，应看 TPOT、输出 Token、并发 Decode、模型大小和投机解码。

关联知识点：

TPOT、投机解码、流式输出、Token ROI。

13. TTFT

难度：中级

建议时长：9 分钟

动画优先级：高

一句话定义：

TTFT 是从用户发出请求到收到第一个 Token 的时间，是用户感知“有没有反应”的关键指标。

要讲清楚：

1. TTFT 包含排队、路由、Prefill、首 Token 生成
2. TTFT 高不等于模型整体慢
3. RAG、工具调用、长上下文都会拉高 TTFT
4. 首字时延对交互体验影响极大
5. 企业平台必须按场景拆解 TTFT

动画素材：

请求链路时间轴：网关、路由、检索、Prefill、首 Token。

企业案例：

Chat Agent 比传统问答体验好，可能不是总时长极低，而是更早流式返回、链路更短、上下文组织更有效。

诊断题方向：

用户反馈“点了半天没反应”，平台总耗时还可以，应优先看 TTFT 而不是平均完成时间。

关联知识点：

Prefill、流式输出、RAG Pipeline、SLA。

14. TPOT

难度：中级

建议时长：8 分钟

动画优先级：中

一句话定义：

TPOT 是每个输出 Token 的平均生成时间，是衡量持续生成速度的重要指标。

要讲清楚：

1. TPOT 影响长文本输出速度
2. TPOT 和 TTFT 是不同指标
3. Decode、模型大小、并发和显存带宽都会影响 TPOT
4. 输出长度控制可以改善总体耗时
5. 不同任务对 TPOT 敏感度不同

动画素材：

展示输出 Token 一个个生成，并标出每个 Token 间隔。

企业案例：

客服问答只输出 100 字，TPOT 影响不大；报告生成输出 3000 字，TPOT 成为主瓶颈。

诊断题方向：

首字很快但用户等完整答案很久，应优先看 TPOT、输出长度和 Decode 并发。

关联知识点：

Decode、投机解码、量化、SLA。

15. KV Cache

难度：高级

建议时长：12 分钟

动画优先级：高

一句话定义：

KV Cache 是缓存历史上下文的 Key 和 Value，避免模型在多轮生成中重复计算全部历史。

要讲清楚：

1. KV Cache 来自注意力机制
2. Prefill 阶段写入 KV Cache
3. Decode 阶段复用 KV Cache
4. KV Cache 占用显存
5. Session 亲和影响缓存命中率
6. 长上下文会挤占 KV Cache 空间

动画素材：

展示输入 Token 进入模型，生成 K/V 缓存槽；下一轮请求命中缓存与未命中的对比。

企业案例：

高峰期 TTFT 从 800ms 涨到 4s，KV Cache 命中率从 60% 降到 15%，优先排查 Session 亲和和路由策略。

诊断题方向：

多轮对话变慢且缓存命中率下降，优先排查请求是否被打散到不同实例。

关联知识点：

Session 亲和、P-D 分离、上下文窗口、MaaS 网关。

16. Session 亲和

难度：高级

建议时长：10 分钟

动画优先级：中

一句话定义：

Session 亲和是把同一会话的连续请求尽量路由到持有其上下文缓存的实例上。

要讲清楚：

1. 为什么多轮对话需要亲和
2. 亲和和负载均衡之间的矛盾
3. 亲和失败会导致 KV Cache 失效
4. 会话迁移需要成本
5. 企业 MaaS 必须观测亲和命中率

动画素材：

展示同一用户多轮请求命中同一实例，以及被打散后的重新 Prefill。

企业案例：

网关升级后负载更均衡，但多轮对话 TTFT 变差。原因可能是均衡策略破坏了缓存亲和。

诊断题方向：

扩容后平均资源利用率更好，但用户首字变慢，应排查亲和策略。

关联知识点：

KV Cache、模型网关、负载均衡、SLA。

17. Batch 调度

难度：高级

建议时长：11 分钟

动画优先级：高

一句话定义：

Batch 调度是把多个请求组织在一起推理，以提升硬件利用率和系统吞吐。

要讲清楚：

1. Batch 提升吞吐但可能增加等待
2. Prefill Batch 和 Decode Batch 特征不同
3. 请求长度不一致会带来调度浪费

- 4. Continuous Batching 为什么重要
- 5. 低时延和高吞吐之间存在取舍

动画素材：

展示多个请求排队后被组成 Batch，同步处理；再展示长短请求混合造成空转。

企业案例：

内部 MaaS 为追求吞吐把 Batch 做大，导致交互式应用首字时延上升。需要按场景分池。

诊断题方向：

吞吐提升但在线交互体验变差，应检查 Batch 大小、排队时间和业务隔离。

关联知识点：

TTFT、TPOT、SLA、模型服务分池。

18. P-D 分离

难度：高级

建议时长：12 分钟

动画优先级：高

一句话定义：

P-D 分离是把 Prefill 和 Decode 两类负载分开调度或部署，以提升资源利用率和服务稳定性。

要讲清楚：

- 1. Prefill 和 Decode 的计算特征不同
- 2. Prefill 更像大块输入处理
- 3. Decode 更像持续小步生成
- 4. 混跑会相互干扰
- 5. 分离后可针对不同阶段优化资源

动画素材：

一条请求拆成 Prefill 队列和 Decode 队列，分别进入不同资源池。

企业案例：

高并发下长上下文请求拖慢短问答请求。P-D 分离和业务分池可以降低互相影响。

诊断题方向：

短请求被长上下文请求拖慢，应考虑 P-D 分离、优先级队列和资源隔离。

关联知识点：

Prefill、Decode、Batch 调度、SLA。

19. 投机解码

难度：高级

建议时长：10 分钟

动画优先级：中

一句话定义：

投机解码是用小模型提前生成候选 Token，再由大模型验证，从而加速生成。

要讲清楚：

1. 为什么 Decode 慢
2. 小模型如何先猜多个 Token
3. 大模型如何批量验证
4. 命中率决定收益
5. 不是所有任务都适合投机解码

动画素材：

小模型一次提出 5 个候选 Token，大模型验证前 4 个通过、第 5 个回退。

企业案例：

通用问答收益较高，但复杂代码生成收益不稳定，因为小模型猜测不容易被大模型接受。

诊断题方向：

开启投机解码后收益低，应看接受率、任务类型、小模型质量和验证开销。

关联知识点：

Decode、TPOT、小模型路由、成本优化。

20. 量化

难度：中级

建议时长：10 分钟

动画优先级：中

一句话定义：

量化是用更低精度表示模型参数或计算过程，以降低显存占用和提升推理效率。

要讲清楚：

1. FP16、INT8、INT4 的基本含义
2. 量化降低显存和带宽压力
3. 量化可能损失精度
4. 不同模型和任务对量化敏感度不同
5. 企业部署要在成本、速度、质量间取舍

动画素材：

展示同一个权重从高精度数字压缩成低精度表示，显存占用减少但误差增加。

企业案例：

内部部署 27B 模型时，量化后单卡可承载更多并发，但复杂推理任务准确率下降，需要评测集验证。

诊断题方向：

量化后简单问答没问题，代码任务明显变差，应检查任务敏感度和模型评测。

关联知识点：

Eval、模型路由、成本治理、Roofline。

模块 03：模型怎么变成企业平台

21. MaaS

难度：中级

建议时长：10 分钟

动画优先级：中

一句话定义：

MaaS 是把模型能力以统一、稳定、可治理的服务方式提供给业务使用的平台形态。

要讲清楚：

1. MaaS 不只是 API 转发
2. 它要提供统一接入、路由、计量、限流、审计
3. 它是成本控制面
4. 它是稳定性控制面
5. 它是企业 AI 治理入口

动画素材：

多个业务应用通过 MaaS 访问内部模型、外部模型、缓存、审计和计量模块。

企业案例：

各团队自行接外部 API，成本不可控、权限不可审、质量不可比。统一 MaaS 后可以集中路由、缓存和观测。

诊断题方向：

外部 API 成本突然上升，应优先看网关计量、缓存命中、模型路由和业务调用结构。

关联知识点：

模型网关、多模型路由、Token ROI、审计。

22. 模型网关

难度：中级

建议时长：10 分钟

动画优先级：高

一句话定义：

模型网关是业务应用访问模型的统一入口，负责协议适配、路由、鉴权、限流、计量和观测。

要讲清楚：

1. 为什么不能让业务直连模型
2. 网关如何屏蔽不同模型 API 差异
3. 网关如何记录调用和成本
4. 网关如何执行限流、熔断和降级
5. 网关如何成为治理抓手

动画素材：

请求进入网关后依次经过鉴权、路由、缓存、模型调用、审计、返回。

企业案例：

业务团队从内部模型切换到外部模型，如果没有网关，需要改大量代码；有网关则只改路由策略。

诊断题方向：

多个应用接入模型后成本和故障难以定位，应判断是否缺统一网关。

关联知识点：

MaaS、计量审计、限流熔断、多模型路由。

23. 多模型路由

难度：高级

建议时长：11 分钟

动画优先级：高

一句话定义：

多模型路由是根据任务类型、成本、时延、质量和可用性，把请求分配给合适模型的能力。

要讲清楚：

1. 为什么不能所有任务都用最大模型
2. 路由维度包括能力、成本、延迟、稳定性
3. 简单任务走小模型，复杂任务走大模型
4. 内部模型优先，外部模型补位
5. 路由策略需要评测和观测闭环

动画素材：

不同请求被分类后路由到小模型、大模型、代码模型、外部模型。

企业案例：

FAQ、摘要、分类走内部小模型；复杂代码修复和深度规划走外部强模型，整体成本下降且质量可控。

诊断题方向：

大模型调用成本居高不下，应看任务分布、路由策略、模型能力边界和缓存命中。

关联知识点：

能力路由、成本路由、Eval、Token ROI。

24. 成本路由

难度：高级

建议时长：10 分钟

动画优先级：中

一句话定义：

成本路由是在满足质量要求的前提下，把请求优先分配给成本更低的模型或路径。

要讲清楚：

1. 成本不只是模型单价
2. 成本包括输入、输出、缓存、重试和失败成本
3. 便宜模型失败重试可能更贵

4. 成本路由必须有质量底线
5. 企业应做按场景的成本预算

动画素材：

同一个任务走小模型一次成功、大模型一次成功、小模型失败后升级大模型三种成本对比。

企业案例：

问题单规范化可以用小模型；复杂代码修复必须用强模型，否则反复失败导致总成本更高。

诊断题方向：

成本路由后质量下降、重试增加，应评估端到端成本，而不是只看单次 Token 单价。

关联知识点：

多模型路由、Token ROI、Eval、重试幂等。

25. 能力路由

难度：高级

建议时长：10 分钟

动画优先级：中

一句话定义：

能力路由是根据任务对模型能力的要求，把请求路由到具备相应推理、代码、长上下文或工具能力的模型。

要讲清楚：

1. 不同模型能力不同
2. 能力标签需要通过评测建立
3. 路由前需要任务识别
4. 能力不足会造成隐性失败
5. 能力路由比成本路由更靠近质量底线

动画素材：

请求先被打上“代码修复、摘要、检索问答、规划”等标签，再进入对应模型。

企业案例：

代码审查任务用通用小模型会漏掉跨文件影响，必须路由到代码能力更强的模型。

诊断题方向：

低成本模型在复杂任务上输出看似合理但不可用，应判断为能力路由失败。

关联知识点：

推理能力边界、Eval、多模型路由、质量门禁。

26. 缓存体系

难度：高级

建议时长：11 分钟

动画优先级：高

一句话定义：

缓存体系是在模型调用链路中复用可复用结果，减少重复计算、降低成本并提升响应速度。

要讲清楚：

1. 缓存不只有 KV Cache
2. 还有前缀缓存、语义缓存、工具结果缓存、检索结果缓存
3. 缓存命中需要稳定输入和良好键设计
4. 缓存可能带来过期和权限问题
5. 企业平台要按场景设计缓存层次

动画素材：

请求经过前缀缓存、工具缓存、检索缓存、模型调用缓存的命中与未命中路径。

企业案例：

相同系统提示词和相同工具说明反复出现在 Prompt 中，可用前缀缓存降低 TTFT 和成本。

诊断题方向：

缓存命中率低，应检查输入是否稳定、上下文是否有随机噪声、缓存键是否合理、权限是否隔离。

关联知识点：

KV Cache、Token ROI、RAG、模型网关。

27. 限流熔断

难度：中级

建议时长：10 分钟

动画优先级：中

一句话定义：

限流和熔断是防止模型服务被突发流量或异常调用拖垮的稳定性机制。

要讲清楚：

1. 限流控制进入系统的请求量
2. 熔断在下游异常时主动停止调用
3. 不同业务需要不同优先级
4. 限流要结合配额和降级
5. 熔断后要有用户可理解的反馈

动画素材：

高峰流量进入网关后，普通请求排队，低优先级请求被限流，高优先级请求保留。

企业案例：

某团队脚本误调用导致 token 暴增，如果没有限流，会影响全平台用户。

诊断题方向：

平台突然拥塞，应检查是否有异常调用方、是否限流、是否按租户隔离。

关联知识点：

SLA、计量审计、模型网关、权限治理。

28. SLA 保障

难度：中级

建议时长：10 分钟

动画优先级：低

一句话定义：

SLA 保障是围绕可用性、时延、错误率和容量承诺，设计模型平台的稳定性体系。

要讲清楚：

1. AI 应用 SLA 不只是接口成功率
2. TTFT、TPOT、P95/P99 都是关键指标
3. 模型质量波动也是广义 SLA
4. 需要降级、容灾、灰度和回滚
5. 不同业务应有不同 SLA 分级

动画素材：

展示请求从正常路径切换到降级模型、备用模型或排队路径。

企业案例：

内部研发工具可以接受短时降级，一线客户支撑工具必须优先保障可用性和稳定性。

诊断题方向：

模型 API 可用但用户认为不可用，应检查首字时延、回答质量、工具失败率和端到端链路。

关联知识点：

TTFT、限流熔断、Observability、模型降级。

模块 04：模型怎么变成 Agent

29. Prompt 与 Context

难度：基础

建议时长：9 分钟

动画优先级：中

一句话定义：

Prompt 是用户或系统给模型的显式指令，Context 是模型当前可见的全部信息环境。

要讲清楚：

1. Prompt 只是 Context 的一部分
2. Context 包括系统指令、历史、工具结果、文档、代码
3. Agent 成败更多取决于上下文工程
4. 上下文越多不等于越好
5. 企业 Agent 需要结构化上下文

动画素材：

展示一个请求中系统提示词、用户问题、历史记录、检索文档、工具结果共同组成上下文。

企业案例：

同样模型，传统问答效果差，Chat Agent 效果好，可能是上下文组织方式完全不同。

诊断题方向：

模型答非所问，不能只改 Prompt，要检查完整 Context。

关联知识点：

上下文窗口、上下文污染、AGENTS.md、RAG。

30. 系统提示词

难度：基础

建议时长：8 分钟

动画优先级：低

一句话定义：

系统提示词是高优先级指令，用于定义模型的角色、边界、行为准则和输出约束。

要讲清楚：

1. 系统提示词比用户提示词更稳定
2. 它适合放角色、边界、安全和格式要求
3. 不应塞入大量业务知识
4. 系统提示词过长会增加成本和干扰
5. 系统提示词要版本化和评测

动画素材：

展示相同用户问题在不同系统提示词下输出不同风格和边界。

企业案例：

Code Review Agent 的系统提示词应规定审查重点、输出格式、禁止直接改代码等行为边界。

诊断题方向：

Agent 输出格式经常漂移，应检查系统提示词、示例、输出 schema 和评测。

关联知识点：

规格驱动开发、质量门禁、Agent 安全。

31. 上下文窗口

难度：基础

建议时长：9 分钟

动画优先级：高

一句话定义：

上下文窗口是模型一次推理中能看到的最大 Token 范围。

要讲清楚：

1. 模型只能基于窗口内信息回答
2. 超出窗口的信息不可见或需要压缩
3. 长上下文带来成本、时延和注意力稀释

- 4. 大窗口不等于强记忆
- 5. Agent 需要上下文选择机制

动画素材：

展示窗口容量条，新的内容进入，旧内容被压缩、裁剪或摘要。

企业案例：

仓库级代码分析不能把全部代码塞进窗口，需要索引、检索和文件级定位。

诊断题方向：

模型忽略早期约束，应检查是否被窗口挤出、是否压缩丢失、是否优先级过低。

关联知识点：

上下文压缩、RAG、代码库索引、Token。

32. 上下文压缩

难度：中级

建议时长：10 分钟

动画优先级：中

一句话定义：

上下文压缩是在保留任务关键信息的前提下，减少上下文 Token 占用。

要讲清楚：

- 1. 为什么需要压缩
- 2. 摘要、裁剪、结构化提炼的区别
- 3. 压缩可能丢失细节
- 4. 不同信息压缩策略不同
- 5. 压缩结果需要可追溯

动画素材：

长对话被压缩成任务目标、已知事实、决策、待办、风险五类结构化摘要。

企业案例：

多轮 Issue 修复中，Agent 需要保留已修改文件、失败测试、当前假设，而不是保留全部对话原文。

诊断题方向：

压缩后 Agent 重复犯错，应检查摘要是否丢失失败历史和关键约束。

关联知识点：

上下文窗口、Agent Loop、Trace、记忆。

33. 上下文污染

难度：中级

建议时长：10 分钟

动画优先级：中

一句话定义：

上下文污染是无关、过期、冲突或低质量信息进入上下文后，干扰模型判断的现象。

要讲清楚：

1. 污染源包括旧结论、错误日志、无关文档、低质量历史
2. 模型可能无法自动识别信息可信度
3. 污染会导致幻觉和错误决策
4. 需要来源、时间、优先级和可信度标注
5. Agent 要有上下文清洗机制

动画素材：

上下文中混入旧日志和新日志，模型错误关注旧日志；清洗后关注最新证据。

企业案例：

RAG 返回旧版本制度，模型按旧制度回答，造成错误业务建议。

诊断题方向：

答案引用过期内容，应检查召回排序、更新时间过滤和上下文优先级。

关联知识点：

RAG、Answer Grounding、Trace、知识更新。

34. 分层会话

难度：高级

建议时长：10 分钟

动画优先级：低

一句话定义：

分层会话是把复杂任务拆成规划层、执行层、验证层等不同上下文空间，降低相互干扰。

要讲清楚：

1. 单一长会话容易混乱
2. 规划、执行、验证关注的信息不同
3. 子会话可以隔离局部任务
4. 父会话负责汇总状态和决策
5. 多 AI 协作需要分层会话

动画素材：

主会话拆出代码分析子会话、测试修复子会话、审查子会话，最后汇总。

企业案例：

复杂需求开发中，主 Agent 做任务编排，子 Agent 分别做 UI、后端、测试、代码审查。

诊断题方向：

多 Agent 协作互相覆盖结论，应设计分层会话和汇总协议。

关联知识点：

Subagent、多 Agent 协作、对抗性审查。

35. AGENTS.md

难度：中级

建议时长：10 分钟

动画优先级：低

一句话定义：

AGENTS.md 是放在代码仓或目录中的 Agent 指令文件，用于告诉 AI 如何理解、修改和验证当前工程。

要讲清楚：

1. 它是仓库级上下文入口
2. 可以定义构建、测试、代码规范、目录说明
3. 可以降低重复提示成本
4. 多目录可以有不同指令
5. 它是工程化上下文资产

动画素材：

Agent 进入仓库后先读取 AGENTS.md，再按指令运行测试和修改代码。

企业案例：

All in one codebase 里，AGENTS.md 能帮助 Agent 快速理解模块边界、测试命令、代码规范和提交要求。

诊断题方向：

Agent 总是不知道怎么跑测试，应检查是否有 AGENTS.md 或等效工程指令。

关联知识点：

仓库上下文、规格驱动开发、Code Review Agent。

36. 仓库上下文

难度：高级

建议时长：11 分钟

动画优先级：中

一句话定义：

仓库上下文是 Agent 理解一个代码库所需的目录结构、依赖关系、规范、历史和测试体系。

要讲清楚：

1. 仓库边界就是上下文边界
2. 文件、Issue、需求、测试、文档应尽量收敛
3. Agent 需要文件级定位和依赖图
4. 多仓库会增加上下文断裂
5. All in one codebase 能提升 Agent 可用性

动画素材：

展示代码、Issue、需求、测试、计划在同一个仓库内形成上下文闭环。

企业案例：

代码在 A 仓、需求在 B 系统、Issue 在 C 系统，Agent 很难形成完整任务理解。

诊断题方向：

Agent 改了局部代码但破坏其他模块，应检查依赖图、测试覆盖和仓库上下文。

关联知识点：

AGENTS.md、Issue Fix Agent、质量门禁。

37. 规格驱动开发

难度：高级

建议时长：11 分钟

动画优先级：低

一句话定义：

规格驱动开发是先把需求转成清晰、可验证、可拆解的规格，再交给人或 Agent 执行。

要讲清楚：

1. Agent 不适合直接接含糊需求
2. 规格要包含目标、范围、约束、验收标准
3. 规格是任务拆解和测试生成的基础
4. 规格质量决定 Agent 产出质量
5. 企业软件工程要从口头需求转向规格资产

动画素材：

模糊需求被转换成用户故事、接口约束、验收条件、测试用例。

企业案例：

“优化体验”无法交给 Agent；“将 TTFT 从 4s 降到 1.5s，保持准确率不下降”才可执行。

诊断题方向：

Agent 反复做偏，应检查需求是否被规格化，而不是只让模型重试。

关联知识点：

需求拆解 Agent、测试生成 Agent、质量门禁。

38. Agent Loop

难度：中级

建议时长：12 分钟

动画优先级：高

一句话定义：

Agent Loop 是 Agent 围绕目标持续观察、思考、行动、校验并决定是否继续的执行循环。

要讲清楚：

1. Agent 不只是单轮回答
2. Loop 包括目标、观察、计划、行动、校验、停止
3. 工具调用让 Agent 能影响外部系统
4. 退出条件和失败处理非常关键
5. 企业 Agent 必须可追踪、可回滚、可审批

动画素材：

Observe → Think → Act → Check → Continue/Stop 的循环。

企业案例：

Issue Fix Agent 需要理解问题单、定位文件、修改代码、跑测试、修复失败、生成 PR，而不是一次性回答。

诊断题方向：

Agent 无限循环修复失败，应检查退出条件、最大轮次、回滚点和人工审批点。

关联知识点：

工具调用、Trace、Human-in-the-loop、Issue Fix Agent。

39. 工具调用

难度：中级

建议时长：10 分钟

动画优先级：高

一句话定义：

工具调用是让模型通过受控接口查询信息、执行操作或访问外部系统的机制。

要讲清楚：

1. 模型本身不能直接访问系统
2. 工具把模型连接到搜索、文件、数据库、代码、工单系统
3. 工具需要 schema、权限、参数校验
4. 工具结果会进入上下文
5. 工具失败要可重试、可降级

动画素材：

模型判断需要工具，生成参数，调用工具，接收结果，再继续推理。

企业案例：

运维 Agent 不能凭空判断故障，必须调用监控、日志、变更系统。

诊断题方向：

模型编造数据，应检查是否缺工具调用或工具结果未注入上下文。

关联知识点：

Agent Loop、权限治理、工具结果缓存。

40. Skill

难度：高级

建议时长：12 分钟

动画优先级：高

一句话定义：

Skill 是把某类任务的指令、资源、脚本和流程封装成可复用能力单元，让 Agent 可以按需调用。

要讲清楚：

1. Skill 不是普通提示词模板
2. Skill 包含触发条件、操作步骤、资源和脚本
3. Skill 让经验可沉淀、可复用、可版本化
4. Skill 是 Chat as Skill 的基础
5. 企业需要 Skill 生命周期治理

动画素材：

用户一次完成工作流后，沉淀为 Skill；后续相似任务自动触发 Skill。

企业案例：

湖南标杆项目沉淀 60+ Skill，如果触发条件和输入输出不清晰，复用率会低。

诊断题方向：

Skill 数量很多但复用率低，应检查颗粒度、触发条件、输入输出、版本治理。

关联知识点：

Chat as Skill、Agent Loop、Skill Designer、知识资产。

41. Subagent

难度：高级

建议时长：10 分钟

动画优先级：中

一句话定义：

Subagent 是由主 Agent 派生或调用的专门子 Agent，用于处理特定子任务。

要讲清楚：

1. 主 Agent 负责目标和编排
2. Subagent 负责专门任务
3. 子任务上下文可以隔离

4. 结果需要汇总和校验
5. 子 Agent 过多会增加协调成本

动画素材：

主 Agent 拆分任务，分别派给代码分析、测试生成、文档更新、审查 Subagent。

企业案例：

一个复杂 PR 由实现 Agent 修改代码，审查 Agent 找风险，测试 Agent 补用例，主 Agent 汇总。

诊断题方向：

多个 Subagent 输出冲突，应设计仲裁、优先级和汇总协议。

关联知识点：

分层会话、多 Agent 协作、对抗性审查。

42. 记忆

难度：中级

建议时长：10 分钟

动画优先级：低

一句话定义：

记忆是 Agent 在当前上下文之外保存和复用用户偏好、任务状态、历史经验或长期知识的能力。

要讲清楚：

1. 记忆不同于上下文窗口
2. 短期记忆、长期记忆、任务记忆不同
3. 记忆需要写入、检索、更新和遗忘机制
4. 错误记忆会持续污染后续任务
5. 企业场景必须管控记忆权限和边界

动画素材：

一次任务中的关键偏好被写入记忆，下次相似任务被检索出来。

企业案例：

Agent 记住某团队代码规范，下次 Review 自动按规范检查；但如果规范更新，旧记忆必须失效。

诊断题方向：

Agent 一直沿用过期偏好，应检查记忆更新和失效机制。

关联知识点：

上下文压缩、权限治理、Agent 安全。

43. Human-in-the-loop

难度：中级

建议时长：10 分钟

动画优先级：低

一句话定义：

Human-in-the-loop 是在 Agent 执行链路中设置人类确认、审批、纠偏和兜底节点。

要讲清楚：

1. 企业 Agent 不能盲目全自动
2. 高风险操作必须人审
3. 人类应介入目标、权限、合入、发布等关键点
4. 审批点太多会降低效率
5. 要按风险等级设计人机协同

动画素材：

Agent 自动完成低风险步骤，在代码合入和生产操作前等待人类确认。

企业案例：

Issue Fix Agent 可以自动生成 PR，但合入主干和发布必须人审。

诊断题方向：

Agent 自动修改配置导致事故，应检查权限边界和审批点。

关联知识点：

Agent 安全、权限治理、质量门禁。

44. 多 Agent 协作

难度：高级

建议时长：12 分钟

动画优先级：中

一句话定义：

多 Agent 协作是多个具备不同角色和能力的 Agent 围绕同一目标分工、协同、审查和汇总的工作模式。

要讲清楚：

1. 多 Agent 不是简单并发多个模型
2. 需要角色分工、上下文隔离、通信协议
3. 需要主控、审查和仲裁
4. 对抗性审查可以提升质量
5. N 版本竞标适合高价值决策

动画素材：

三个 Agent 分别给方案，审查 Agent 质疑，主 Agent 选择最终方案。

企业案例：

复杂架构设计可让多个模型分别出方案，再由审查 Agent 找漏洞，最后由 Owner 决策。

诊断题方向：

多 Agent 输出很多但无法收敛，应检查是否缺主控和评判标准。

关联知识点：

Subagent、分层会话、对抗性审查、N 版本竞标。

模块 05：Agent 怎么改变软件工程

45. Code Review Agent

难度：高级

建议时长：11 分钟

动画优先级：中

一句话定义：

Code Review Agent 是使用模型和工具自动审查代码变更，发现缺陷、风险、规范问题和测试缺口的 Agent。

要讲清楚：

1. 它不是简单代码风格检查
2. 需要理解 diff、上下文、依赖和测试
3. 输出应聚焦高价值问题
4. 要避免大量低质量评论
5. 人审和 Agent 审查应互补

动画素材：

PR diff 进入 Agent，Agent 读取相关文件、测试和规范，输出风险清单。

企业案例：

稳定的 Code Review Agent 可以作为质量防护网的一层，在人审前筛掉明显问题。

诊断题方向：

Agent 评论很多但开发不采纳，应检查准确率、严重度分级和评论噪声。

关联知识点：

质量门禁、仓库上下文、AGENTS.md、对抗性审查。

46. Issue Fix Agent

难度：高级

建议时长：12 分钟

动画优先级：高

一句话定义：

Issue Fix Agent 是从问题单理解、定位、修改、测试到生成 PR 的自动化修复 Agent。

要讲清楚：

1. 问题单质量决定修复成功率
2. Agent 需要补全复现路径
3. 定位和验证比写代码更重要
4. 自动修复必须配合测试和人审
5. 闭环数据要回流到问题单质量改进

动画素材：

Issue → 规范化 → 定位文件 → 修改代码 → 跑测试 → 生成 PR → 人审。

企业案例：

口语化问题单缺少复现步骤、预期结果和严重级别，Agent 需要先做问题单规范化。

诊断题方向：

Agent 修复失败率高，应先看 Issue 质量，而不是只换更强模型。

关联知识点：

问题单规范化、测试生成 Agent、质量回流。

47. 需求拆解 Agent

难度：高级

建议时长：11 分钟

动画优先级：中

一句话定义：

需求拆解 Agent 是把模糊需求转成规格、任务、验收标准和执行计划的 Agent。

要讲清楚：

1. 需求天然含糊
2. Agent 需要识别目标、范围、约束和风险
3. 拆解结果要能进入 Issue、代码和测试
4. 人类 Owner 负责确认优先级和边界
5. 好的拆解能显著提升后续 Agent 成功率

动画素材：

模糊需求被拆成 Epic、Story、Task、验收标准和测试点。

企业案例：

“做一个 AI 学习 APP” 要拆成首页、模块页、知识点详情、动画组件、诊断题、搜索、进度。

诊断题方向：

开发 Agent 做出不符合预期的功能，应检查需求拆解和验收标准。

关联知识点：

规格驱动开发、Human-in-the-loop、价值复盘。

48. 测试生成 Agent

难度：中级

建议时长：10 分钟

动画优先级：中

一句话定义：

测试生成 Agent 是根据需求、代码和缺陷自动生成单元测试、集成测试、回归测试和验收用例的 Agent。

要讲清楚：

1. 测试生成不是只补覆盖率
2. 要基于风险和行为生成测试
3. 需要结合需求和历史缺陷

4. 生成测试后必须运行和筛选
5. 测试是 Agent 自动修复的安全网

动画素材：

需求和代码输入后，Agent 生成正常路径、异常路径、边界路径测试。

企业案例：

Issue Fix Agent 修复问题后，测试生成 Agent 根据缺陷补充回归用例，防止再次出现。

诊断题方向：

Agent 修了 bug 但下次又复现，应检查是否补了回归测试。

关联知识点：

Issue Fix Agent、质量门禁、Eval。

49. 运维诊断 Agent

难度：高级

建议时长：11 分钟

动画优先级：高

一句话定义：

运维诊断 Agent 是结合监控、日志、链路、变更和知识库，自动分析故障原因并给出处理建议的 Agent。

要讲清楚：

1. 运维诊断必须基于实时证据
2. 日志、指标、Trace、变更是核心上下文
3. Agent 要区分现象、假设、证据和结论
4. 高风险操作要人审
5. 诊断结果要回流知识库

动画素材：

告警进入 Agent，Agent 拉取监控、日志、变更记录，形成诊断链路。

企业案例：

MaaS TTFT 突增，Agent 需要关联流量、Batch、KV Cache、实例异常和最近变更。

诊断题方向：

Agent 只根据历史经验判断故障，应检查是否接入实时工具和证据链。

关联知识点：

工具调用、Observability、Trace、SLA。

50. 价值复盘 Agent

难度：中级

建议时长：10 分钟

动画优先级：低

一句话定义：

价值复盘 Agent 是对需求、问题单、PR、测试、上线和业务指标进行复盘，判断 AI 工作是否产生真实价值的 Agent。

要讲清楚：

1. AI 产出不等于业务价值
2. 要看节省人天、质量提升、时延下降、成本下降
3. 复盘要连接任务、代码、测试、用户反馈
4. Token 消耗要和结果绑定
5. 复盘结果应反哺路由、流程和组织考核

动画素材：

一个需求从立项到上线，Agent 汇总投入、Token、缺陷、周期和收益。

企业案例：

某 Agent 消耗大量 Token 生成代码，但人审大量返工，实际 ROI 很低。

诊断题方向：

AI 使用量上升但业务效果不明显，应建立价值复盘指标。

关联知识点：

Token ROI、Agent 杠杆、组织阵型。

模块 06：企业怎么治理 AI

51. Eval

难度：高级

建议时长：12 分钟

动画优先级：中

一句话定义：

Eval 是用系统化评测集和指标衡量模型、Agent 或 AI 应用在目标任务上的表现。

要讲清楚：

1. Eval 不是随便试几个问题
2. 要有黄金集、场景集、回归集
3. 指标包括准确率、稳定性、成本、时延、人工采纳率
4. Agent 评测比模型评测更复杂
5. Eval 是模型路由和质量门禁基础

动画素材：

同一任务集分别跑模型 A、模型 B、Agent 流程，输出质量、成本、时延对比。

企业案例：

引入新模型前，必须跑 Code Review、Issue Fix、RAG 问答等场景评测，而不是只看通用榜单。

诊断题方向：

新模型上线后用户反馈变差，应检查是否缺场景回归评测。

关联知识点：

模型路由、质量门禁、灰度发布。

52. Trace

难度：高级

建议时长：10 分钟

动画优先级：高

一句话定义：

Trace 是记录 AI 应用从输入、上下文、模型调用、工具调用到输出的完整执行轨迹。

要讲清楚：

1. 没有 Trace 就无法诊断 Agent
2. Trace 要记录 Prompt、Context、工具、模型、耗时、Token
3. Trace 支持问题复现和责任定位
4. Trace 涉及隐私和权限
5. Agent 平台必须默认可追踪

动画素材：

一次 Agent 任务展开成时间线：用户输入、检索、模型调用、工具调用、输出。

企业案例：

Issue Fix Agent 生成错误 PR，Trace 可以还原它读了哪些文件、调用了哪些工具、为什么做出决策。

诊断题方向：

线上错误无法复现，应检查 Trace 是否完整记录上下文和工具结果。

关联知识点：

Observability、Agent Loop、审计。

53. Observability

难度：高级

建议时长：11 分钟

动画优先级：中

一句话定义：

Observability 是通过指标、日志、Trace 和事件，让 AI 平台和 Agent 应用可观测、可诊断、可优化。

要讲清楚：

1. AI 系统观测不只是接口日志
2. 要看 TTFT、TPOT、Token、缓存命中、错误率、工具失败率
3. Agent 还要看步骤成功率和人工接管率
4. 观测要能按租户、模型、场景切片
5. 观测结果要驱动优化

动画素材：

一次请求的数据被拆成链路指标、模型指标、工具指标、业务指标。

企业案例：

MaaS 看平均时延没问题，但某业务 P99 极差。需要按业务和模型分维度观测。

诊断题方向：

用户反馈慢但平台平均值正常，应看 P95/P99、分场景指标和链路 Trace。

关联知识点：

TTFT、TPOT、SLA、Trace。

54. Token ROI

难度：高级

建议时长：12 分钟

动画优先级：高

一句话定义：

Token ROI 是衡量 AI 消耗的 Token 是否转化为有效产出、效率提升或业务价值的指标体系。

要讲清楚：

1. Token 是成本，不是价值
2. 有效 Token 和无效 Token 要区分
3. 调试、重试、失败、冗余上下文都会拉低 ROI
4. ROI 要和节省人天、质量提升、周期缩短绑定
5. 企业平台要按用户、应用、场景计算 ROI

动画素材：

一次 Agent 任务消耗 Token，拆分为有效推理、上下文冗余、工具失败、重试浪费，最后映射到节省人天。

企业案例：

Claude Code 用户大量消耗 Token，但如果能减少 2 小时代码定位和修复，ROI 可能很高；如果只是反复试错，ROI 可能很低。

诊断题方向：

Token 使用量暴涨，应判断是业务增长、上下文膨胀、缓存失效还是失败重试。

关联知识点：

成本路由、缓存体系、价值复盘 Agent。

55. 权限治理

难度：高级

建议时长：11 分钟

动画优先级：中

一句话定义：

权限治理是控制用户、Agent、工具和模型在什么条件下可以访问什么数据、执行什么操作。

要讲清楚：

1. Agent 权限不能等同于用户权限无限继承
2. 需要最小权限原则

3. 工具调用要有参数和动作边界
4. 高风险操作要审批
5. 权限日志要可审计

动画素材：

用户请求经过身份、场景、工具、数据权限检查后，部分操作被允许，部分被拦截。

企业案例：

运维 Agent 可以查询日志，但不能直接重启生产服务；代码 Agent 可以生成 PR，但不能自动合入主干。

诊断题方向：

Agent 误操作生产系统，应检查权限边界、审批点和工具能力暴露。

关联知识点：

Human-in-the-loop、Agent 安全、审计。

56. AI 原生组织阵型

难度：高级

建议时长：12 分钟

动画优先级：低

一句话定义：

AI 原生组织阵型是围绕 Owner、小队、Agent 军团、质量门禁和平台治理重构软件生产方式。

要讲清楚：

1. 人不再只是写代码，而是定义目标、规格、上下文和验收
2. 最小作战单元变成 Owner + 工程师 + 质量 Owner + N 个 Agent
3. 中间协调工作会被工具吸收
4. 新角色包括 Skill Designer、Agent Operator、AI Quality Owner
5. 考核要从工作量转向价值闭环和 Agent 杠杆

动画素材：

传统串行组织变成 Owner 指挥多个 Agent 小队并行完成需求、代码、测试、运维。

企业案例：

一个 10 人团队借助 Agent 编队支撑 30 人以上的产出，但前提是有清晰规格、上下文资产和质量门禁。

诊断题方向：

团队上了很多 AI 工具但产能没明显提升，应检查组织阵型、工作流、质量门禁和责任边界。

关联知识点：

价值复盘 Agent、Human-in-the-loop、Skill、质量门禁。

4. 内容开发优先级

P0：必须优先做精的 16 讲

这些是 APP 的招牌内容，必须有高质量动画和诊断题。

1. Token
2. 注意力机制
3. 自回归生成
4. Prefill
5. Decode
6. TTFT
7. KV Cache
8. Batch 调度
9. P-D 分离
10. 模型网关
11. 多模型路由
12. 缓存体系
13. 上下文窗口
14. Agent Loop
15. Skill
16. Token ROI

P1：第二批做扎实的 24 讲

这些支撑完整体系。

1. 词向量与语义空间
2. Transformer
3. 采样策略
4. 幻觉
5. 推理能力边界
6. Session 亲和
7. 投机解码
8. 量化
9. MaaS
10. 成本路由
11. 能力路由
12. SLA 保障
13. Prompt 与 Context
14. 上下文压缩
15. 上下文污染
16. AGENTS.md
17. 仓库上下文

18. 规格驱动开发
19. 工具调用
20. Subagent
21. Issue Fix Agent
22. 运维诊断 Agent
23. Eval
24. Trace

P2：第三批补齐的 16 讲

这些用于增强完整性和长期学习价值。

1. 位置编码
2. 指令微调与偏好优化
3. TPOT
4. 限流熔断
5. 系统提示词
6. 分层会话
7. 记忆
8. Human-in-the-loop
9. 多 Agent 协作
10. Code Review Agent
11. 需求拆解 Agent
12. 测试生成 Agent
13. 价值复盘 Agent
14. Observability
15. 权限治理
16. AI 原生组织阵型

5. 动画类型映射

animationType 建议枚举

```
type AnimationType =  
  | "token-flow"  
  | "semantic-space"  
  | "attention-map"  
  | "context-window"  
  | "prefill-decode"  
  | "kv-cache"  
  | "batch-scheduler"  
  | "pd-separation"  
  | "model-router"  
  | "cache-layers"  
  | "rag-pipeline"  
  | "agent-loop"  
  | "tool-calling"  
  | "skill-lifecycle"
```

```
| "issue-fix-flow"  
| "observability-trace"  
| "token-roi-flow";
```

推荐首版先做的 8 个动画组件

1. token-flow
2. attention-map
3. context-window
4. prefill-decode
5. kv-cache
6. model-router
7. agent-loop
8. issue-fix-flow

这 8 个组件可以覆盖 56 讲里大部分重点内容。不要为每一讲都写独立动画，应该通过配置复用。

6. 诊断题开发原则

诊断题不要考定义，要考工程判断。

错误示例：

KV Cache 是什么？

正确示例：

某 MaaS 平台高峰期 TTFT 从 800ms 上升到 4s，KV Cache 命中率从 60% 下降到 15%。用户反馈多轮对话首字变慢。你优先排查什么？

每道题必须包含：

1. 企业场景
2. 关键现象
3. 干扰选项
4. 正确判断
5. 排查路径
6. 关联知识点

7. 给内容开发 AI 的总提示词

你是“AI 应用工程学习 APP”的课程内容开发 Agent。

你的任务是根据指定的知识点，生成一篇适合企业 AI 应用负责人学习的课程内容。

内容不是科普文章，不是论文综述，不是营销文案，也不是提示词技巧合集。

目标读者具备软件工程和 AI 应用背景，但需要把关键知识点系统化掌握到能解释机制、判断方案、诊断问题、指导落地。

每一讲必须包含以下结构：

1. 一句话定义
2. 为什么重要
3. 心智模型
4. 机制拆解
5. 动画演示脚本
6. 企业案例
7. 常见误区
8. 诊断题
9. 核心结论
10. 关联知识点

写作要求：

1. 中文输出
2. 保留必要英文技术名词，例如 Token、Agent、Skill、MaaS、RAG、TTFT、TPOT、KV Cache
3. 不要写空泛口号
4. 不要堆术语
5. 每个机制都要能落到工程现象
6. 每个企业案例都要能落到 MaaS、RAG、Agent、软件工程闭环、成本治理或组织阵型
7. 诊断题必须像真实企业问题，而不是考试选择题
8. 每一讲最终要能转成结构化 JSON 数据
9. 不要生成页面 UI 代码
10. 不要改动课程目录，除非明确要求

输出格式：

课程标题

一句话定义

为什么重要

心智模型

机制拆解

动画演示脚本

动画目标

步骤 1

步骤 2

步骤 3

步骤 4

企业案例

常见误区

诊断题

场景

问题

选项

正确答案

解析

排查路径

核心结论

关联知识点

8. 推荐开发节奏

第一轮：先做 12 讲样板

优先做：

1. Token
2. 注意力机制
3. Prefill
4. Decode
5. TTFT
6. KV Cache
7. 模型网关
8. 多模型路由
9. 上下文窗口
10. Agent Loop
11. Skill
12. Issue Fix Agent

目标：

把内容结构、动画风格、诊断题质量跑通。

第二轮：扩到 28 讲

补齐 P0 和部分 P1。

目标：

形成完整主线，能够支撑 MVP 演示。

第三轮：扩到 56 讲

补齐全部课程。

目标：

形成完整 AI 应用工程教材。

9. 最重要的取舍

这 56 讲的重点不是“知识覆盖面最大”，而是“企业 AI 应用工程闭环最完整”。

不要把课程做成：

1. 大模型百科
2. Prompt 教程
3. AI 新闻合集
4. 算法公式课
5. 工具使用手册

要做成：

1. 模型机制课
2. 推理系统课
3. MaaS 平台课
4. Agent 工程课
5. 软件工程变革课
6. 企业治理课

最终用户看完后，应该能回答：

1. 模型为什么慢？
2. 为什么 TTFT 高？
3. 为什么 RAG 答不准？
4. 为什么 Agent 会跑偏？
5. 为什么 Skill 复用率低？
6. 为什么 Token 成本失控？
7. 为什么 AI 工具很多但组织效率没提升？
8. 为什么企业需要 MaaS、Eval、Trace 和质量门禁？